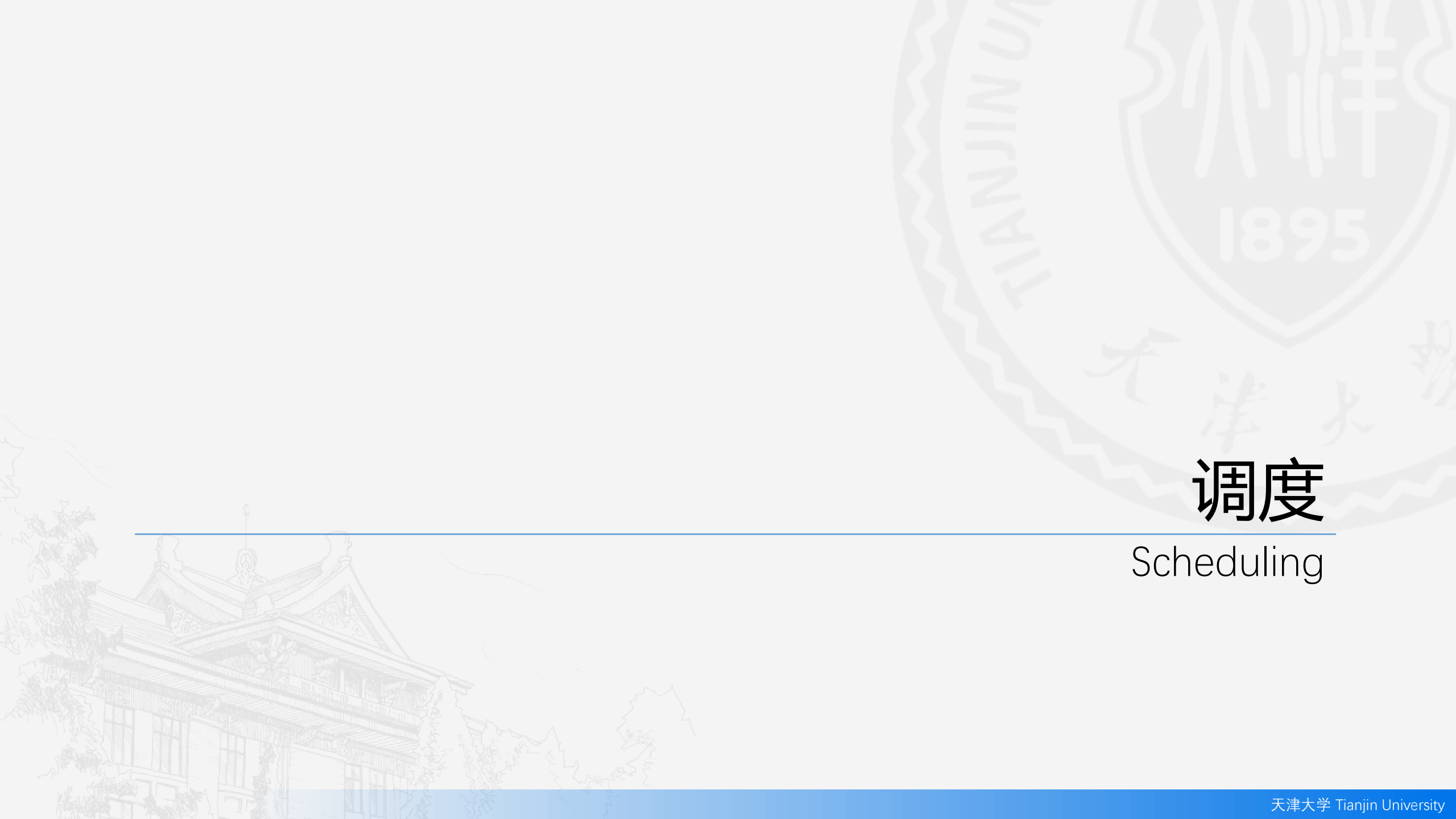




# 调度

Scheduling



## 内容介绍

- 基本概念 Basic Concepts
- 调度准则 Scheduling Criteria
- 调度算法 Scheduling Algorithms



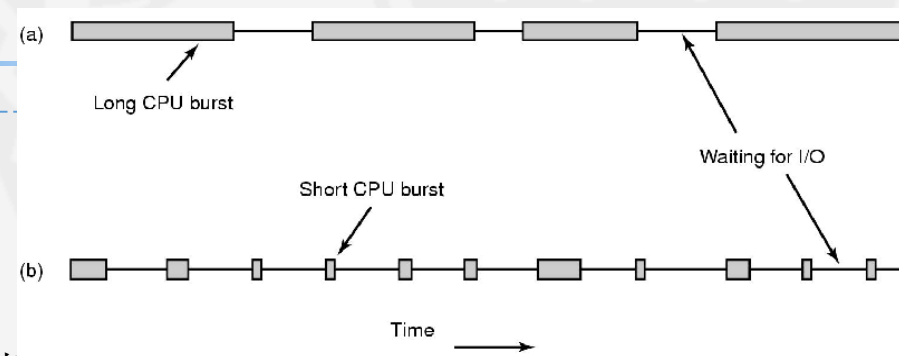
## 调度的引入

- 早期的批处理系统中，I/O 设备和 CPU 是串行工作的。操作系统中只有一个可执行程序，一个任务执行时，必须 CPU 执行完，I/O 才能执行，两者不可并行。因此，当 I/O 执行时，昂贵的 CPU 处理能力被白白浪费。
- 而且，随着CPU速度迅速提高而I/O设备速度却提高不多，导致 CPU 和 I/O 设备之间的速度不匹配的矛盾越来越突出。为了解决 CPU 的处理能力被浪费的问题，提出了多道程序的概念。
- **多道程序设计 (Multi-programming)**：允许多个程序（作业）同时进入一个计算机系统的内存并启动进行交替计算的方法，也就是**并发 (concurrency)**，计算机中可以同时存放多道程序，从宏观上来看它们是并行的，多道程序都同时处于运行过程中，都未运行结束，但是微观上是串行的，轮流占用CPU交替执行。
- **多道程序设计的目标**，是无论何时都有进程运行，从而最大化CPU利用率，充分发挥计算机系统部件的并行性。所以，系统中会有多个进程或线程同时竞争CPU。
- **分时系统的目标**，是在进程之间快速切换 CPU，以便用户在程序运行时能与其交互。
- 为了满足这些目标，进程**调度程序 (scheduler)** 选择一个可用进程到CPU上执行。单处理器系统不会具有多个正在运行的进程。如果有多个进程，那么其余的需要等待CPU空闲并能重新调度。



## 进程的类型

- 大多数进程可以分为 I/O 密集型进程 和 CPU 密集型进程。
- **CPU 密集型进程 (CPU-bound process)**
  - 很少产生 I/O 请求，将主要时间用于使用 CPU 执行计算。
  - 例如计算圆周率、对视频进行高清解码等，全靠CPU的运算能力。
  - 对于CPU密集型进程，进程越多，花在进程切换的时间就越多，CPU执行任务的效率就越低。
- **I/O 密集型进程 (I/O-bound process)**
  - 执行 I/O 比使用 CPU 执行计算需要花费更多时间。这类进程的特点是CPU消耗很少，进程的大部分时间都在等待I/O操作完成（因为I/O的速度远远低于CPU和内存的速度）。
  - 涉及到网络、磁盘I/O的进程都是I/O密集型进程。常见的大部分进程都是I/O密集型进程，比如Web应用。
  - 对于I/O密集型进程，进程越多，CPU效率越高，但也有一个限度。
- 随着CPU变得越来越快，更多的进程倾向为I/O密集型。



(a) CPU密集型进程 (b) I/O密集型进程



## 调度类型

- 在操作系统中，调度是指将进程或线程分配给CPU进行处理的过程。根据调度的频率和目的，调度可以分为长期调度、中期调度和短期调度。
- **长期调度 (Long-term Scheduling)**，又称为**高级调度**、**作业调度 (Job Scheduling)**。
  - 对于**批处理系统**，提交的进程多于可以立即执行的。这些进程会被保存到大容量存储设备的缓冲池中。长期调度程序从缓冲池中选择进程，加到内存，以便执行。
  - 作用：控制系统中的**多道程序程度 (Degree of Multiprogramming)**，决定哪些新进程可以进入系统。
  - **多道程序程度 (Degree of Multiprogramming)**：内存中的进程数量。
  - 平衡资源使用：为了让 CPU 和 I/O 设备都保持忙碌状态，长期调度器会选择合适的 CPU-bound 和 I/O-bound 进程的组合进入系统，以充分利用系统资源。
  - **较少触发**：仅在创建新进程或系统负载发生显著变化时触发。这种调度发生不频繁，因为它主要决定哪些进程可以被“引入”到系统中。
  - 对于**分时系统**，如Windows、Unix、Linux等，一般没有长期调度程序，只是简单的将所有新进程都放入内存，以供短期调度程序使用。





## 调度类型

- **中期调度 (Medium-term Scheduling)**，又称为**中级调度**、**内存调度**、**交换调度**。
  - 作用：控制多道程序程度和内存利用：主要负责内存管理，决定哪些进程需要被**交换 (Swapping)**到内存外，以缓解内存压力或调整多道程序的数量。
  - **交换 (Swapping)**：当系统内存不足或系统负载过高时，中期调度器可以将某些进程暂时从主内存中移出（交换到硬盘），以腾出空间给其他进程。
  - 触发：根据内存压力触发。当系统内存紧张或需要调整多道程序程度时触发。这样可以确保系统在内存使用方面保持平衡，提高系统的响应速度。
  - 目的：提高内存利用率，维持系统的响应性，确保在内存资源有限的情况下，进程可以顺利执行。
- 我们将在后面的内存管理中详细讨论。



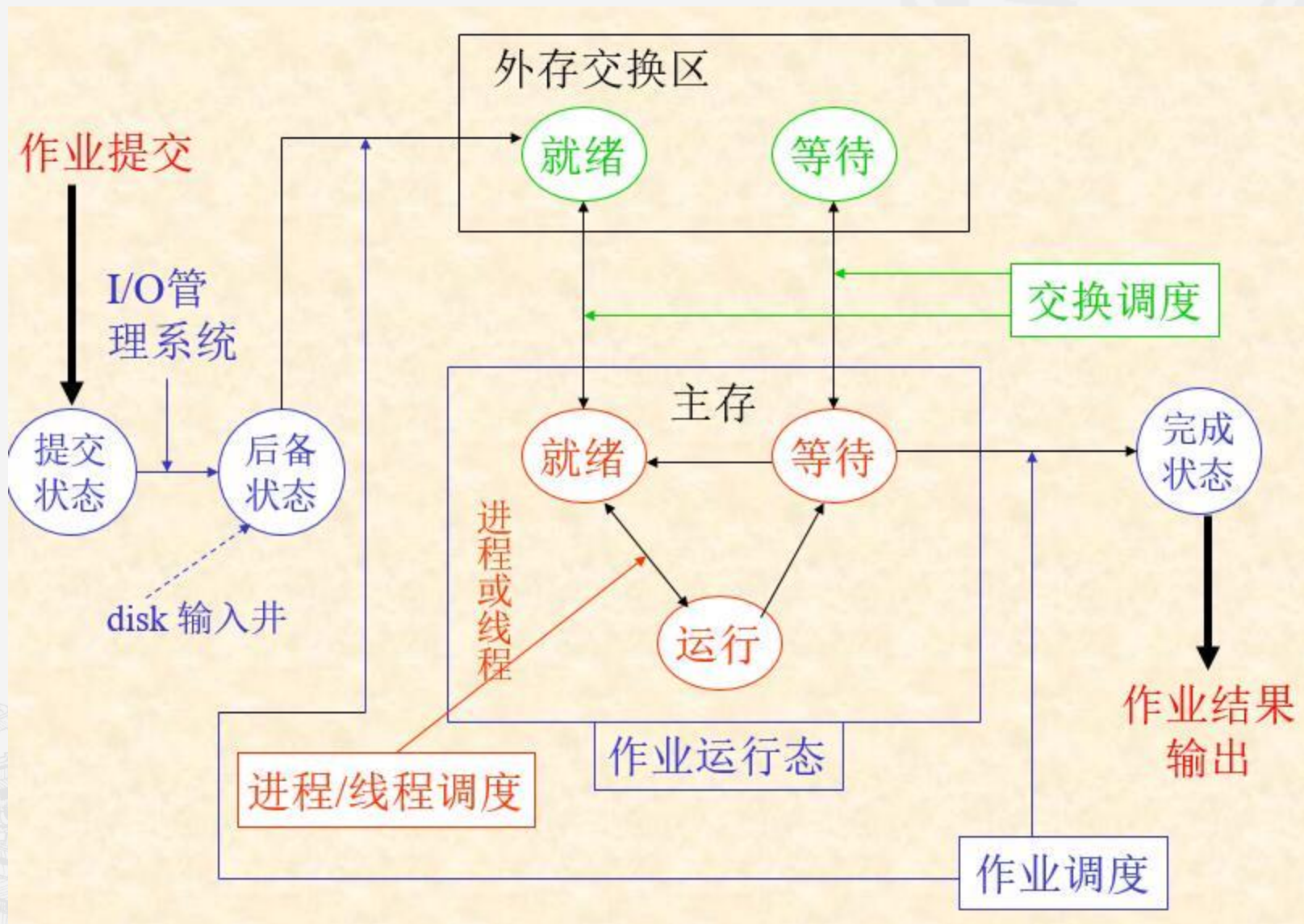
## 调度类型

- **短期调度 (Short-term Scheduling)**，又称为**低级调度**、**进程调度 (Process Scheduling)**、**CPU调度 (CPU Scheduling)**
  - 作用：管理**就绪队列 (Ready Queue)**：负责选择就绪队列中的下一个进程并将其分配给 CPU 执行。
  - **就绪队列 (Ready Queue)**：驻留在内存中的，处于就绪状态，等待运行的进程保存在就绪队列中。
  - 短期调度器（也叫**调度器 Scheduler** 或**分派器 Dispatcher**）是进程调度中最频繁的一部分。
  - 响应各种事件：
    - 时钟中断 (Clock interrupts)：例如，在抢占式系统中，当一个时间片到期时，调度器会被触发。
    - I/O 中断 (I/O interrupts)：当 I/O 操作完成时，调度器选择下一个进程。
    - 系统调用 (System calls)：当进程发出阻塞调用时，调度器决定哪个进程应立即获得 CPU。
    - 信号量操作 (Semaphore operations)：当资源变得可用时，调度器会唤醒等待中的进程。
  - **触发：非常频繁**。在进程调度中触发频率最高。通常，短期调度程序每100ms至少执行一次，因此必须高效快速。
  - 目的：最小化响应时间。快速选择和分配进程，确保系统保持高效运转，提供及时的响应。



# 调度

Scheduling







## CPU调度程序

- 每当CPU空闲时，操作系统就必须从**就绪队列**中选择一个进程来执行。进程的选择由**短期调度程序**即**CPU调度程序**执行。
- 就绪队列不必是先进先出（FIFO）队列，队列中的记录通常为进程控制块（PCB）。
- CPU调度决策可在如下四种环境下发生：
  - 1. 当一个进程从运行状态切换到等待状态（例如：I/O请求）
  - 2. 当一个进程从运行状态切换到就绪状态（例如：出现中断）
  - 3. 当一个进程从等待状态切换到就绪状态（例如：I/O完成）
  - 4. 当一个进程终止时
- 当调度只能发生在第1和第4两种情况下时，称调度方案是**非抢占的（nonpreemptive）**或**协作的（cooperative）**；否则，称调度方案是**抢占的（preemptive）**。采用非抢占调度，一旦CPU分配给一个进程，那么该进程会一直使用CPU，直到进程终止或切换到等待状态。



## CPU调度程序

- CPU 调度器 Scheduler 或分派器 Dispatcher：分派程序是一个模块，用来将CPU的控制权交给由短期调度程序选择的进程。其功能包括：
  - 切换上下文
  - 切换到用户模式
  - 跳转到用户程序的合适位置，以重新启动程序
- 调度程序应尽可能快，因为在每次进程切换时都要使用。
- 调度延迟（dispatch latency）：调度程序停止一个进程而启动另一个所需的时间。



## 调度准则

- **CPU利用率 (CPU Utilization)**：CPU的繁忙程度
- **吞吐量 (Throughput)**：它指一个时间单元内所完成进程的数量。
- **周转时间 (Turnaround time)**：从进程提交到进程完成的时间段称为周转时间。
- **等待时间 (Waiting time)**：等待时间为在就绪队列中等待所花时间之和。
- **执行时间 (Executing time)**：进程在运行态，使用CPU执行指令所花时间之和。
- **响应时间 (Response time)**：从提交请求到产生第一响应的的时间
- 对于**批处理系统 (Batch System)**而言，关注的指标有：
  - 吞吐量 (尽可能大)
  - 周转时间 (尽可能短)
  - CPU利用率 (尽可能高)
- 对于**交互式系统 (Interactive System)**而言，关注的指标为：
  - 响应时间 (对请求要及时响应)



## 调度算法

- CPU调度程序，需要处理的是如何从就绪队列中选择进程并为之分配CPU时间的问题，选择的方法即为调度算法。操作系统有多种不同的CPU调度算法。
- 以下调度算法多用于批处理系统
  - 先到先服务调度 (First-Come, First-Served (FCFS) Scheduling)
  - 最短作业优先调度 (Shortest-Job-First (SJF) Scheduling)
  - 最短剩余时间优先调度 (Shortest-Remaining-Time-First (SRTF) Scheduling)
  - 最高响应比优先调度 (High-Response-Ratio-First (HRRF) Scheduling)
  - 优先级调度 (Priority Scheduling)
- 以下调度算法多用于分时系统
  - 轮转调度 (Round Robin (RR))
  - 多级队列调度 (Multilevel Queue Scheduling)
  - 多级反馈队列调度 (Multilevel Feedback Queue Scheduling)





## 先到先服务调度 (First-Come, First-Served (FCFS) Scheduling)

- **先到先服务调度 (First-Come, First-Served (FCFS) Scheduling)**
- 可以通过 FIFO 队列实现。缺点是平均等待时间往往较长。
- 注意：FCFS调度算法是非抢占的。一旦CPU分配给了一个进程，该进程就会使用CPU直到释放CPU为止，即程序终止或是请求I/O。FCFS算法一般用于批处理系统的作业调度，不适用于分时系统。
- **例题：**已知三个作业依次到达系统，到达时间和预期执行时间如表中所示（单位：分钟）。系统**采用FCFS调度算法**，求作业的平均周转时间。

| 作业号<br>Job ID | 到达时间<br>Arrive Time | 结束时间<br>Finish Time | 等待时间<br>Waiting time | 执行时间<br>Executing time | 周转时间<br>Turnaround time |
|---------------|---------------------|---------------------|----------------------|------------------------|-------------------------|
| J1            | 10:00               |                     |                      | 120                    |                         |
| J2            | 10:10               |                     |                      | 60                     |                         |
| J3            | 10:25               |                     |                      | 25                     |                         |



## 先到先服务调度 (First-Come, First-Served (FCFS) Scheduling)

- 10:00 由于J1先到达，先执行J1，到12:00结束。注意FCFS不可抢占，J1未执行完前，其他作业如果到达也要等待。
- 12:00 此时J2和J3在等待，按照先到先服务原则，执行J2，到13:00结束。
- 13:00 执行J3，到13:25结束。
- 周转时间 = 等待时间 + 执行时间，也可以计算 到达时间 与 结束时间 之间的距离。
- 平均周转时间 = 各作业周转时间之和 / 作业数 =  $(120+170+180)/3=156.7$  (分)

| 作业号<br>Job ID | 到达时间<br>Arrive Time | 结束时间<br>Finish Time | 等待时间<br>Waiting time | 执行时间<br>Executing time | 周转时间<br>Turnaround time |
|---------------|---------------------|---------------------|----------------------|------------------------|-------------------------|
| J1            | 10:00               | 12:00               | 0                    | 120                    | 120                     |
| J2            | 10:10               | 13:00               | 110                  | 60                     | 170                     |
| J3            | 10:25               | 13:25               | 155                  | 25                     | 180                     |



## 最短作业优先调度 (Shortest-Job-First (SJF) Scheduling)

- 最短作业优先调度 (Shortest-Job-First (SJF) Scheduling)
- 当CPU空闲时，选择（预期）最短执行时间的进程。SJF算法的困难点在于如何知道进程未来的执行时间。对于批处理系统，可以将用户提交作业时指定的进程时限作为执行时间。故SJF算法适用于批处理系统中的作业调度，不适用于短期CPU调度。
- 可以证明SJF调度算法是最优的，对一组进程而言，SJF算法的平均等待时间最短。
- 注意：FCFS调度算法是非抢占的。一旦CPU分配给了一个进程，该进程就会使用CPU直到释放CPU为止，即程序终止或是请求I/O。FCFS调度算法不适用于分时系统。
- SJF算法可以是抢占的或非抢占的。
  - 当一个新进程到达就绪队列而以前进程正在执行时，非抢占SJF算法会允许当前运行的进程先执行完成。
  - 而抢占SJF算法就需要选择，如果新进程可能有一个更短的CPU执行时间，抢占SJF算法可抢占当前运行的进程。抢占SJF调度有时称为**最短剩余时间优先调度 (Shortest-Remaining-Time-First (SRTF) Scheduling)**。



## 最短作业优先调度 (Shortest-Job-First (SJF) Scheduling)

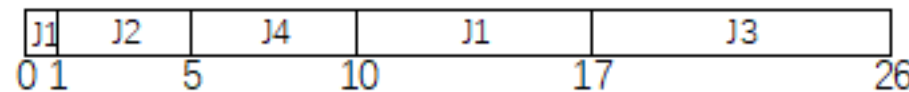
- 题目同前，采用非抢占的SJF算法，计算平均周转时间。
- 10:00 由于J1先到达，先执行J1，到12:00结束。注意：10:00时J1已到达而其他作业未到达，所以不能让已到达的等未到达的。而采用非抢占SJF，J1未执行完前，其他作业如果到达也要等待。
- 12:00 此时J2和J3在等待，按照短作业优先原则，执行J3，到12:25结束。
- 12:25 执行J2，到13:25结束。
- 平均周转时间 =  $(120+195+120)/3=145$  min (分)

| 作业号<br>Job ID | 到达时间<br>Arrive Time | 结束时间<br>Finish Time | 等待时间<br>Waiting time | 执行时间<br>Executing time | 周转时间<br>Turnaround time |
|---------------|---------------------|---------------------|----------------------|------------------------|-------------------------|
| J1            | 10:00               | 12:00               | 0                    | 120                    | 120                     |
| J2            | 10:10               | 13:25               | 135                  | 60                     | 195                     |
| J3            | 10:25               | 12:25               | 95                   | 25                     | 120                     |



## 最短剩余时间优先调度 (Shortest-Remaining-Time-First (SRTF) Scheduling)

- 采用**抢占的SJF算法 (SRTF)**，计算平均周转时间 (=13分)
- 10:00，先执行J1。10:01，J2到达，因预期执行时间较短，J2抢占CPU。10:02，J3到达，J3预期执行时间较长，等待。10:03，J4到达，等待。
- 10:05，J2执行完成，此时J1, J3和J4的剩余执行时间分别为 7, 9, 5，先执行J4。
- 10:10，J4执行完成，此时J1和J3的剩余执行时间分别为 7, 9，先执行J1。10:17，执行J3。



| 作业号<br>Job ID | 到达时间<br>Arrive Time | 结束时间<br>Finish Time | 等待时间<br>Waiting time | 执行时间<br>Executing time | 周转时间<br>Turnaround time |
|---------------|---------------------|---------------------|----------------------|------------------------|-------------------------|
| J1            | 10:00               | 10:17               | 9                    | 8                      | 17                      |
| J2            | 10:01               | 10:05               | 0                    | 4                      | 4                       |
| J3            | 10:02               | 10:26               | 15                   | 9                      | 24                      |
| J4            | 10:03               | 10:10               | 2                    | 5                      | 7                       |





## 最高响应比优先调度 (Highest-Response-Ratio-First (HRRF) Scheduling)

- 先到先服务算法 (FCFS) 调度算法，特点是实现简单，公平性较好。
  - 由于按照作业到达的先后顺序进行调度，所以所有作业都可以得到执行，不存在作业“饥饿”问题。
  - 缺点是如果有长作业在前面，后面的作业就必须等待很长时间才能得到执行，导致平均等待时间较长。
- 最短作业优先算法 (SJF) 虽然性能最优，平均等待时间最短，但是对长作业不友好。
  - 一个持续稳定的短作业到达流，可能造成长作业长时间的无法投入运行（饥饿 starvation）
- 最高响应比优先算法 (HRRF) 介于先到先服务算法 (FCFS) 和最短作业优先算法 (SJF) 之间的一种算法，它既考虑了作业的等待时间，又考虑了作业的处理时间。
- 响应比 = 周转时间/执行时间 = (等待时间+执行时间)/执行时间 = 1 + 等待时间/执行时间
- 调度规则：响应比 (Response Ratio) 最高的作业先运行。
- 优点：HRRF对于作业的时间分配比较均匀，既照顾了短作业，又不至于使长作业等待时间过长。
- 缺点：HRRF每次计算响应比都会花费一定的时间，即时间开销。其性能比SJF算法略差。



## 最高响应比优先调度 (High-Response-Ratio-First (HRRF) Scheduling)

- 采用非抢占的HRRF算法，计算平均周转时间。
- 10:00，先执行J1。其间J2、J3、J4到达，因不可抢占，等待J1执行完成。
- 10:08，J1执行完成。计算J2, J3, J4的响应比为 $R2=2$ ,  $R3=1.8$ ,  $R4=1.75$ 。R2最大，选J2投入运行。
- 10:14，J2执行完成，计算J3, J4的响应比为  $R3=3$ ,  $R4=3.25$ 。R4最大，选J4投入运行。
- 10:18，J4执行完成，执行J3。平均周转时间=13（分）

| 作业号<br>Job ID | 到达时间<br>Arrive Time | 结束时间<br>Finish Time | 等待时间<br>Waiting time | 执行时间<br>Executing time | 周转时间<br>Turnaround time |
|---------------|---------------------|---------------------|----------------------|------------------------|-------------------------|
| J1            | 10:00               | 10:08               | 0                    | 8                      | 8                       |
| J2            | 10:02               | 10:14               | 6                    | 6                      | 12                      |
| J3            | 10:04               | 10:23               | 14                   | 5                      | 19                      |
| J4            | 10:05               | 10:18               | 9                    | 4                      | 13                      |



## 优先级调度 (Priority Scheduling)

### ■ 优先级调度 (Priority Scheduling)

- 每个进程都有一个优先级与之关联，具有最高优先级的进程会分配到CPU。具有相同优先级的进程按FCFS顺序调度。优先级调度可以抢占的或非抢占的。
- SJF就是一个简单的优先级算法，其优先级为预期下次CPU执行时间，时间越短，优先级越高。
- 优先级的定义可能是内部的（如任务使用资源的特点）或外部的（如缴费情况）。
- 优先级通常为固定区间的数字，如0~127。
  - 不同的系统中，数字大小含义不同，有的数字越大，优先级越高，有的则越低。
  - 我们一般采用数字越大优先级越低，即 0 代表优先级最高。
- 优先级调度算法的一个主要问题是“饥饿 (starvation)”
  - 某个低优先级的进程可能无穷等待CPU。
  - 据说，在1973年MIT关闭一台IBM 7094机时，一个1967年提交的低优先级进程尚未得到执行。
  - 解决方案之一是老化 (aging)。例如，得不到执行的进程，每15分钟优先级数字减1。



## 轮转调度 (Round Robin (RR))

- **轮转调度 (Round Robin (RR))**：为分时系统设计，类似FCFS，但增加了**抢占**以切换进程。
  - **时间片 (time slice)**：一个较小的时间单元，一般为  $10 \sim 100\text{ms}$
  - CPU调度程序循环整个就绪队列，为每一个进程分配不超过一个时间片的CPU。
  - 没有进程被连续分配超过一个时间片的CPU，除非它是唯一的可运行进程。
  - 进程的CPU执行时间达到一个时间片而尚未结束时，该进程会被抢占，并被放回就绪队列中。
- RR算法的性能很大程度上取决于时间片的大小。
  - 时间片太大，会导致RR算法和FCFS算法变得一样
  - 时间片太小，会导致大量的进程上下文切换
  - 我们希望时间片远大于上下文切换时间（现代计算机一般 $<10 \mu\text{s}$ ）



## 多级队列调度 (Multilevel Queue Scheduling)

- **多级队列调度 (Multilevel Queue Scheduling)** 算法将就绪队列分成多个单独队列。
  - 每个队列可以有自己的调度算法，比如有的队列使用RR调度，有的队列使用FCFS调度。
  - 队列之间通常采用优先级抢占调度。只有高优先级的队列为空时，才会调度低优先级队列中的就绪进程。
  - 进程进行系统时，永久的分配到某个队列中。这可能导致分配到低优先级队列中的进程长期无法得到执行而导致**饥饿 (starvation)**。
- **多级反馈队列调度 (Multilevel Feedback Queue Scheduling)** 是对上述问题的改进。
  - 根据不同CPU执行特点区分进程，允许进程在不同的队列之间迁移。
  - 如果进程使用过多的CPU时间，会被移到更低优先级的队列。
  - 相应的，I/O密集型和交互进程会放在更高优先级的队列中。
  - 在较低优先级队列中等待过长的进程，会被移到更高优先级队列中，通过**老化 (aging)** 阻止饥饿的发生。





## 多级反馈队列调度 (Multilevel Feedback Queue Scheduling)

- 例：某系统的CPU调度采用多级反馈队列调度算法。
- 共128个就绪队列，队列内采用RR调度，队列间采用抢占的优先级调度，优先级数值范围为0~127，其中0代表最高优先级。只有高优先级的队列为空时，才会调度低优先级队列中的就绪进程。
- 进程分为二种：实时进程和普通进程。实时进程的优先级创建时指定，之后固定不变。普通进程的优先级动态可变。计算方法如下。
- 思考题：1. 应如何设置实时进程的优先级，保证实时进程总是优先于普通进程得到执行？
- 2. 普通进程的优先级计算公式中，调整哪个参数可以保证重要进程优先执行？哪个参数保证了公平，避免了饥饿？
- 3. 为什么每秒要调整所有进程的CPU\_USAGE值？
- 4. 如果想保证系统中CPU密集型进程得到较多的CPU执行时间，应如何调整R和D这两个参数？

普通进程的优先级计算公式： $Priority = base + nice + CPU\_PENALTY$

- Priority：普通进程的优先级，范围40~127，数值越小优先级越高
- base：普通进程优先级的基准值，固定为40
- nice：友好值，范围0~39，前台进程默认值20，后台进程默认值24。使用nice命令可以指定新进程的nice值，使用renice命令调整进程的nice值。普通用户只能增大自己拥有的进程的nice值，只有管理员可以减小。
- CPU\_PENALTY：CPU惩罚值，由另一个参数计算得出。 $CPU\_PENALTY = CPU\_USAGE * R$  (R 取值范围0~1，默认0.5)
- 处于运行态的进程，每次使用CPU一个时间片，该进程的CPU\_USAGE自动加1。时间片取10ms。
- 系统每一秒都会调整所有进程的CPU\_USAGE，方法是  $CPU\_USAGE = CPU\_USAGE * D$  (D 取值范围0~1，默认0.5)



## 多级反馈队列调度 (Multilevel Feedback Queue Scheduling)

- 普通进程的优先级计算公式:  $\text{Priority} = \text{base} + \text{nice} + \text{CPU\_PENALTY}$
- 模拟运行: 系统中没有实时进程, 普通进程只有2个, P1和P2, nice值分别为20和30, 同时就绪后轮流执行。时间片大小为10ms, 所以1秒被分为100个时间片, 由P1和P2轮流使用。
- 开始: P1的优先级 $40+20+0=60$ , P2的优先级 $40+30+0=70$ , 由P1先运行。
- 前20个时间片均由P1使用, 之后P1的优先级 $40+20+10=70$ , 此时与P2优先级相同, 二者开始轮流公平地使用时间片。
- 第1秒内, P1共使用了60个时间片, P2共使用了40个时间片。这体现了nice值小的进程优先, 而且nice值大的进程也不会饥饿。
- 第1秒结束时, 所有进程的CPU\_USAGE减半, 重新计算优先级, P1的优先级 $40+20+15=75$ , P2的优先级 $40+30+10=80$ , 仍然由P1先使用CPU, 随后与P2轮流公平使用CPU。



## 多级反馈队列调度 (Multilevel Feedback Queue Scheduling)

■ 普通进程的优先级计算公式:  $\text{Priority} = \text{base} + \text{nice} + \text{CPU\_PENALTY}$

■ 思考题:

■ 1. 应如何设置实时进程的优先级, 保证实时进程总是优先于普通进程得到执行?

答: 实时进程的优先级应取0~39之间的固定值, 因为普通进程的优先级范围为40~127。

■ 2. 普通进程的优先级计算公式中, 调整哪个参数可以保证重要进程优先执行? 哪个参数保证了公平, 避免了饥饿?

答: 可以适当减小重要进程的nice值, 使其优先执行。CPU\_PENALTY保证了调度的公平, 避免了饥饿。

■ 3. 为什么每秒要调整所有进程的CPU\_USAGE值?

答: 不每秒调整所有进程的CPU\_USAGE值的话, 所有普通进程的优先级都会逐渐变成127。

■ 4. 如果想保证系统中CPU密集型进程得到较多的CPU执行时间, 应如何调整R和D这两个参数?

答: 可将参数R调小。极端情况设置R=0, 相当于对使用CPU的情况不记录不惩罚, 对CPU密集型进程有利。

参数D的情况则比较复杂, 将参数D调小与调小参数R类似, 极端情况D=0, 相当于每秒对CPU使用记录清0。

但是调大参数D的值, 会导致所有进程的优先级最后都上升到127, 相当于不惩罚使用CPU较多的进程了。



## 课后阅读

- 《操作系统概念 OSC（第9版）》第5章：5.1节 – 5.3节。
- 《现代操作系统 MOS（第4版）》第2章：2.4节。